

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт

Про виконання лабораторної роботи №3
“Мова програмування Сі, компіляція, функції вводу/виводу”

Виконала:

Студентка ФЕП-13с

Пономаренко Д.С.

Перевірив

проф. Крупич О. М.

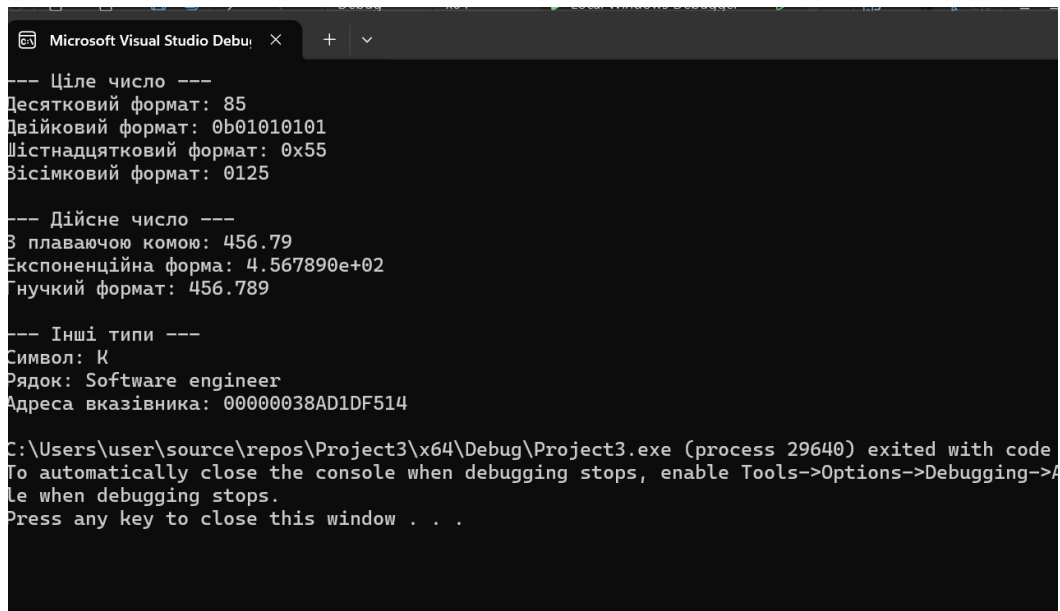
Львів 2026

Мета роботи:

Вивчити поняття: компіляції, функцій вводу/виводу.

Хід роботи

- ❖ Опрацювати і засвоїти матеріал наведений в теоретичних відомостях.
- ❖ Написати програму на мові програмування Cі, котра ілюструє роботу функції printf() з використанням основних специфікаторів. Вивести ціле число у десятковому, двійковому, вісімковому і шістнадцятковому форматах. Вивести дійсне число (з дробовою частиною) у формі з плаваючою комою, в експоненційній формі і в гнучкій формі (специфікатор g). Вивести символ, стрічку і вказівник.

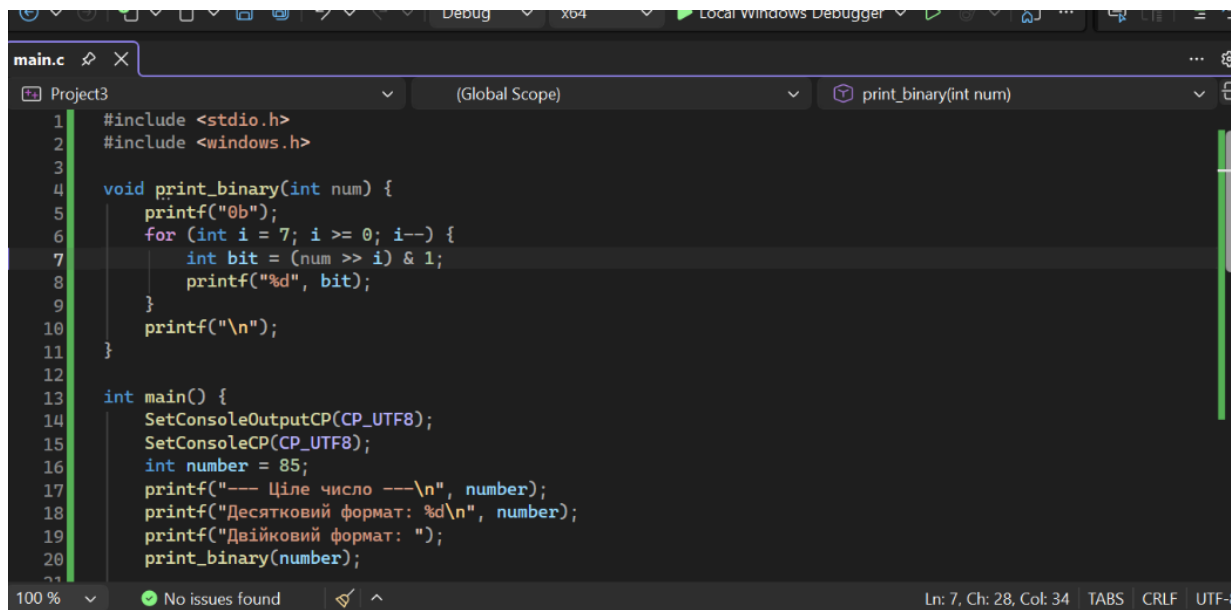


```
--- Ціле число ---
Десятковий формат: 85
Двійковий формат: 0b01010101
Шістнадцятковий формат: 0x55
Вісімковий формат: 0125

--- Дійсне число ---
З плаваючою комою: 456.79
Експоненційна форма: 4.567890e+02
Гнучкий формат: 456.789

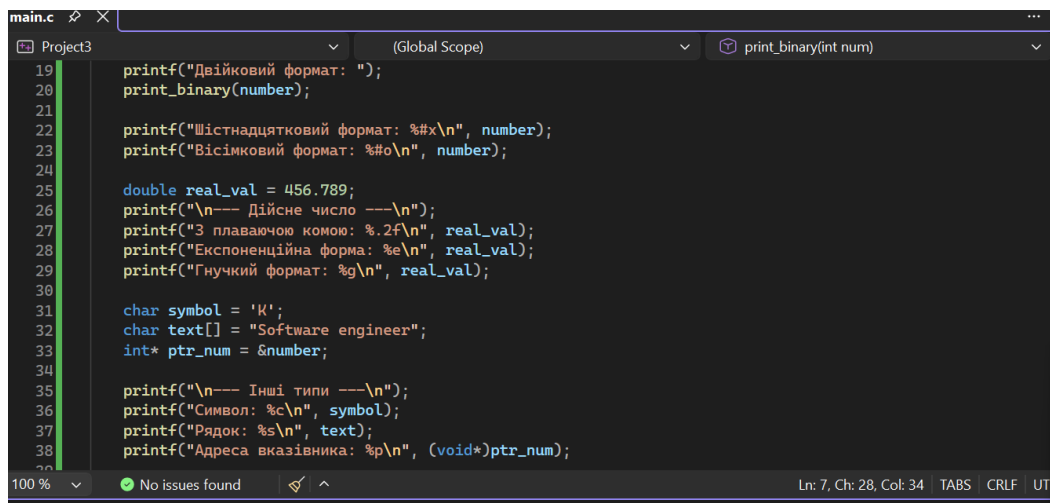
--- Інші типи ---
Символ: K
Рядок: Software engineer
Адреса вказівника: 00000038AD1DF514

C:\Users\user\source\repos\Project3\x64\Debug\Project3.exe (process 29640) exited with code
To automatically close the console when debugging stops, enable Tools->Options->Debugging->A
le when debugging stops.
Press any key to close this window . . .
```



```
main.c
Project3
(Global Scope)
print_binary(int num)

1 #include <stdio.h>
2 #include <windows.h>
3
4 void print_binary(int num) {
5     printf("0b");
6     for (int i = 7; i >= 0; i--) {
7         int bit = (num >> i) & 1;
8         printf("%d", bit);
9     }
10    printf("\n");
11 }
12
13 int main() {
14     SetConsoleOutputCP(CP_UTF8);
15     SetConsoleCP(CP_UTF8);
16     int number = 85;
17     printf("--- Ціле число ---\n", number);
18     printf("Десятковий формат: %d\n", number);
19     printf("Двійковий формат: ");
20     print_binary(number);
21 }
```



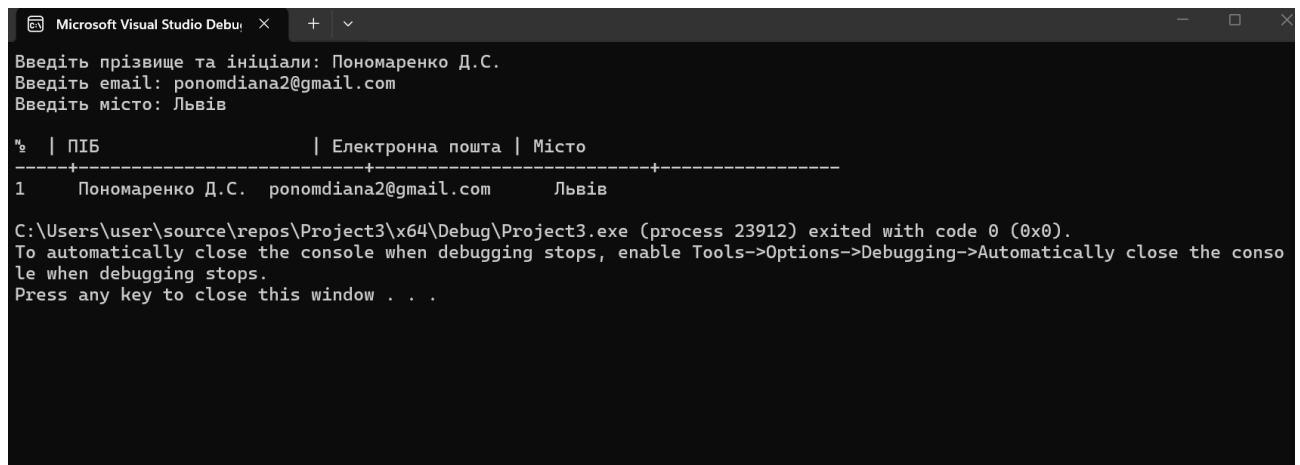
```
main.c
Project3
(Global Scope)
print_binary(int num)

19 printf("Двійковий формат: ");
20 print_binary(number);
21
22 printf("Шістнадцятковий формат: %#x\n", number);
23 printf("Вісімковий формат: %#o\n", number);
24
25 double real_val = 456.789;
26 printf("\n--- Дійсне число ---\n");
27 printf("З плаваючою комою: %.2f\n", real_val);
28 printf("Експоненційна форма: %e\n", real_val);
29 printf("Гнучкий формат: %g\n", real_val);
30
31 char symbol = 'K';
32 char text[] = "Software engineer";
33 int* ptr_num = &number;
34
35 printf("\n--- Інші типи ---\n");
36 printf("Символ: %c\n", symbol);
37 printf("Рядок: %s\n", text);
38 printf("Адреса вказівника: %p\n", (void*)ptr_num);
39
```

Для виводу
цілого числа
number у
двійковій
системі я
написала
окрему
функцію

print_binary, тому що у стандартній бібліотеці C немає готового специфікатора формату для двійкових чисел. У ній я використала цикл від 7 до 0 та побітовий зсув $(num \gg i) \& 1$, щоб похвилинно витягнути кожен із 8 біт байта без додаткових обчислень чи виділення пам'яті. На початку функції main я викликала SetConsoleOutputCP(CP_UTF8) та SetConsoleCP(CP_UTF8) з бібліотеки <windows.h>, щоб консоль Windows коректно відображала українські літери (до цього намагалася написати код без цих команд, але програма не сприймала текст українською). Далі для цілого числа я використала специфікатори %d для десяткового формату, %#x для шістнадцяткового та %#o для вісімкового, де специфікатор # зручний, бо автоматично додає відповідні префікси 0x та 0. Для дійсного числа real_val типу double я використала %.2f для обмеження виводу двома знаками після коми з округленням, %e для демонстрації наукової (експоненційної) форми запису та %g для автоматичного вибору найкоротшого формату. Наприкінці я також показала роботу з іншими типами даних: вивела окремий символ symbol через %c, текстовий рядок text через %s та адресу пам'яті змінної number за допомогою вказівника ptr_num і специфікатора %p.

- ❖ Написати програму котра реалізує введення кількох стрічок символів та виведення їх у складі форматowanego тексту (наприклад, таблиця списку групи з колонками: № п/п; Прізвище, Ініціали; Ел.пошта; Улюблений колір тощо).

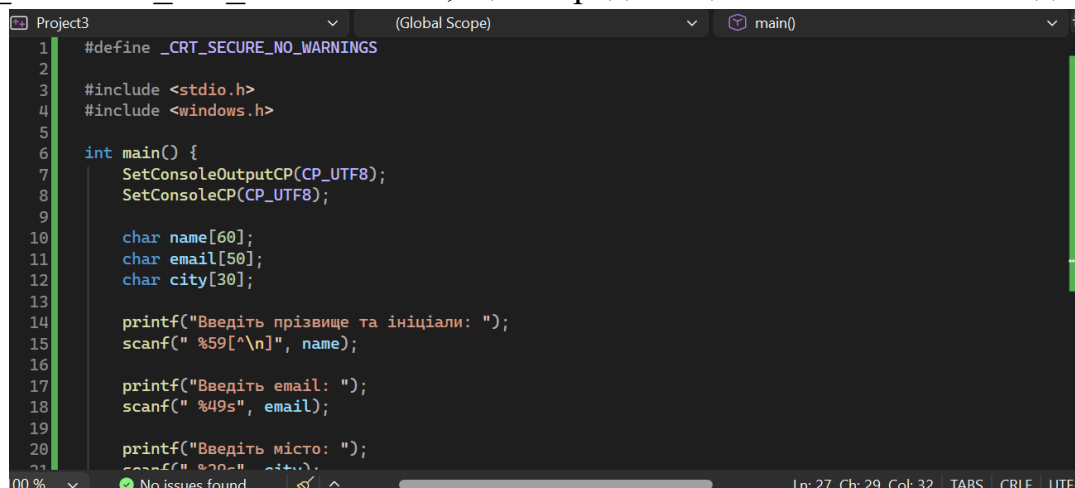


```
Microsoft Visual Studio Debug Console
Введіть прізвище та ініціали: Пономаренко Д.С.
Введіть email: ponomdiana2@gmail.com
Введіть місто: Львів

% | ПІБ | Електронна пошта | Місто
-----+-----+-----+-----
1 | Пономаренко Д.С. | ponomdiana2@gmail.com | Львів

C:\Users\user\source\repos\Project3\x64\Debug\Project3.exe (process 23912) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

У цій програмі я забезпечила форматове введення даних користувача з консолі та їхнє подальше виведення у вигляді структурованої таблиці з вирівняними колонками. На самому початку файлу я додала директиву `#define _CRT_SECURE_NO_WARNINGS`, щоб середовище Visual Studio не видавало



```
Project3 (Global Scope) main()
1 #define _CRT_SECURE_NO_WARNINGS
2
3 #include <stdio.h>
4 #include <windows.h>
5
6 int main() {
7     SetConsoleOutputCP(CP_UTF8);
8     SetConsoleCP(CP_UTF8);
9
10    char name[60];
11    char email[50];
12    char city[30];
13
14    printf("Введіть прізвище та ініціали: ");
15    scanf("%59[^\n]", name);
16
17    printf("Введіть email: ");
18    scanf("%49s", email);
19
20    printf("Введіть місто: ");
21    scanf("%29s", city);
22}
```

помилку при використанні стандартної функції `scanf`. У функції `main` за допомогою `SetConsoleOutputCP(CP_UTF8)` та `SetConsoleCP(CP_UTF8)` я налаштувала кодування UTF-8 для коректного відображення українських літер. Для збереження тексту я оголосила три масиви символів: `name`, `email` та `city`. При зчитуванні прізвища та ініціалів використала специфікатор `scanf("%59[^\n]", name)`, який дозволяє прочитати весь рядок із пробілами до натискання `enter`. Для електронної пошти та міста застосувала `%49s` і `%29s`. В усіх випадках я чітко обмежила кількість символів на 1 менше від розміру масивів, щоб гарантовано уникнути переповнення буфера пам'яті. Для формування таблиці використала специфікатори форматування ширини з мінусом, як-от `%-25s`, що забезпечує рівне вирівнювання тексту по лівому краю

```

16
17     printf("Введіть email: ");
18     scanf("%49s", email);
19
20     printf("Введіть місто: ");
21     scanf("%29s", city);
22
23     printf("\n");
24     printf("%-4s | %-25s | %-25s | %-15s\n", "", "ПІБ", "Електронна пошта", "Місто");
25     printf("-----\n");
26
27     printf("%-4d %-25s %-25s %-15s\n", 1, name, email, city);
28
29     return 0;
30 }

```

всередині кожної
колонки, а між
колонками додала
вертикальні
розділювачі | та
горизонтальну лінію
для чіткої візуальної
структури.

Висновок

Під час виконання лабораторної роботи я засвоїла базові принципи роботи з функціями введення та виведення даних `printf()` і `scanf()` у мові програмування C, а також особливості роботи з ними на Windows.

У першому завданні я навчилася використовувати основні специфікатори формату для виведення цілих та дійсних чисел у різних системах числення (десятковій, шістнадцятковій, вісімковій) і форматах, а також реалізувала власну функцію `print_binary()` для виводу числа у двійковому вигляді за допомогою побітових операцій зсуву та маскування. Додатково опрацювала роботу з символами, рядками та вказівниками пам'яті. У другому завданні я реалізувала зчитування текстових даних від користувача та їхнє відображення у вигляді структурованої таблиці. Я практично застосувала специфікатори ширини з лівим вирівнюванням (наприклад, `%-25s`) для вирівнювання колонок, а також виконала захист від переповнення буфера пам'яті шляхом обмеження кількості символів при зчитуванні за допомогою `scanf()`.